generated by gpt 5.1 mini — Gradient Descent

Foundations, Types, Mathematics, and Practical Tips

A detailed two-column guide explaining gradient descent: theory, formulas, comparison of variants, common pitfalls, and practical enhancements. Includes illustrative images.

Autor: generated by gpt 5.1 mini

February 2026

Introduction

Introduction. Gradient descent is an iterative optimization algorithm used to minimize a cost (loss) function by moving parameters in the direction of the negative gradient. It is fundamental to training many machine learning models, especially neural networks.

Mathematical background

Mathematical background. Key concepts are derivatives (rate of change), gradients (multivariate derivatives), and the cost function J(θ) to be minimized. The gradient points in the direction of steepest increase; descent follows the negative gradient.

$$J(\theta) = \frac{1}{2m}\sum_{i=1}^{m}\left(h_\theta(x^{(i)}) - y^{(i)}\right)^2$$

Mean squared error cost for linear regression.

Vector form and update rule. For parameter vector θ and learning rate α, gradient descent updates are compactly written in vector notation. The method repeats until convergence or a stopping criterion.

$$\theta := \theta - \alpha\nabla_\theta J(\theta)$$

Gradient descent update rule (vector form).

Types of gradient descent

Types of gradient descent. There are three widely used flavors: batch (full) gradient descent, stochastic gradient descent (SGD), and mini-batch gradient descent. They differ in the amount of data used to compute each gradient step.

- Batch (full) gradient descent — uses the entire training set for each update; stable but costly for large datasets.
- Stochastic gradient descent (SGD) — updates parameters per training example; noisy but fast and good for online learning.
- Mini-batch gradient descent — uses small batches (e.g., 32–512 examples); balances noise and efficiency.

Table 1: Comparison of gradient descent variants

Variant	Update frequency	Pros	Cons
Batch	Once per epoch (full dataset)	Stable direction; deterministic	Very slow for large datasets; memory heavy
Stochastic (SGD)	Per example	Fast, online updates, escapes shallow local minima	Very noisy; may need smaller learning rate
Mini-batch	Per small batch	Efficient on hardware, reduced noise	Requires tuning batch size

Algorithm and pseudocode

Algorithm overview. Initialize parameters (often randomly), compute gradients, update parameters using the chosen rule, and repeat. Monitor loss and

optionally apply learning-rate schedules or optimizer variants.

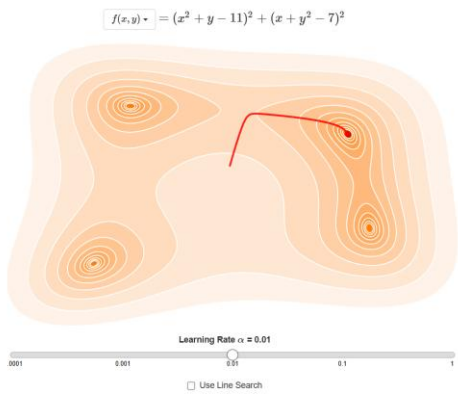

Figure: Gradient descent path on a simple 1D cost surface.

Learning rate and convergence

Learning rate and convergence. The scalar learning rate α controls step size. Too large α can diverge; too small α leads to slow convergence or getting stuck on plateaus. Adaptive methods mitigate this issue.

$$\frac{\partial}{\partial \theta_j} J(\theta) = \frac{1}{m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)}\right) x_j^{(i)}$$

Partial derivative of MSE with respect to parameter \theta_j.

Problems and pitfalls

Common problems and pitfalls. Local minima, saddle points, plateaus, and ill-conditioned (elongated) cost surfaces can slow or trap gradient descent. Proper initialization, momentum, and adaptive optimizers help.

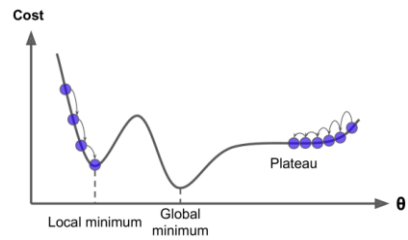

Figure: Common pitfalls — local minima, plateaus, and saddle points.

Practical enhancements

Practical enhancements. Useful techniques include Momentum, Nesterov Accelerated Gradient, RMSProp, Adagrad, and Adam. Regularization, batch-normalization, and learning-rate schedules also improve training.

- Momentum / Nesterov — smooths updates using an exponential moving average of past gradients.
- RMSProp / Adagrad — scale updates by a per-parameter learning rate derived from past squared gradients.
- Adam — combines momentum and RMSProp ideas; widely used as a default optimizer.

Table 2: Popular optimizer methods

Optimizer	Key idea	Typical use
Momentum	Accelerates in consistent gradient directions	Large-scale deep networks
RMSProp	Adaptive per-parameter step sizes from squared gradients	Recurrent networks, non-stationary objectives
Adam	Momentum + RMSProp (bias-corrected)	Default choice for many problems

Using gradient descent in practice

Using gradient descent in practice. Choose batch size, learning rate, and optimizer based on data size and model. Monitor training/validation loss and adjust hyperparameters; use early stopping to avoid overfitting.

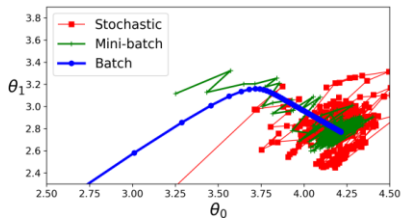

Figure: Comparison of trajectories for Batch, Mini-batch, and Stochastic gradient descent.

Conclusion

Conclusion. Gradient descent is a versatile optimization technique. Understanding its math and practical variants lets you train models more effectively and diagnose convergence issues.

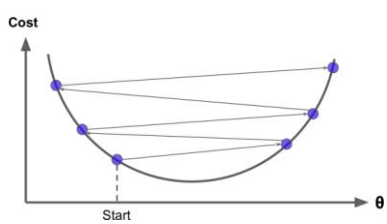

Figure: Large learning rate vs small learning rate behavior (illustrative).

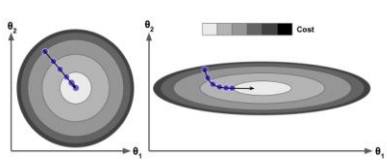

Figure: Gradient descent visualized in two features (contour + path).

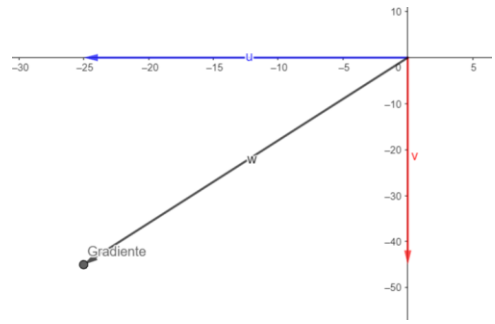

Figure: 3D view of a cost surface; gradient direction shown.

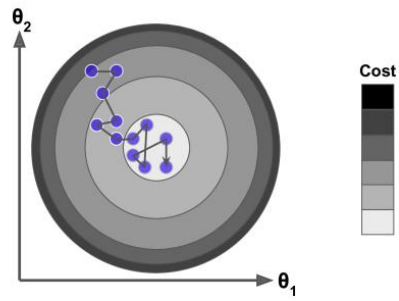

Figure: Stochastic gradient descent steps (noisy path).

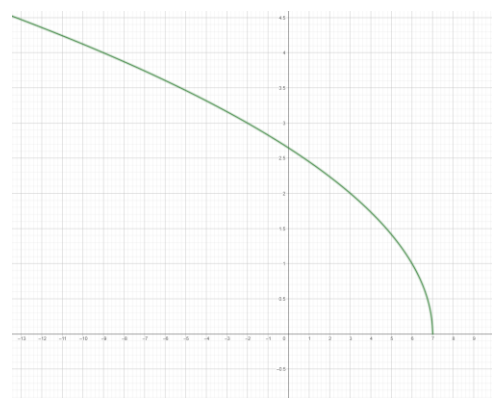

Figure: Domain and function visualization (example).

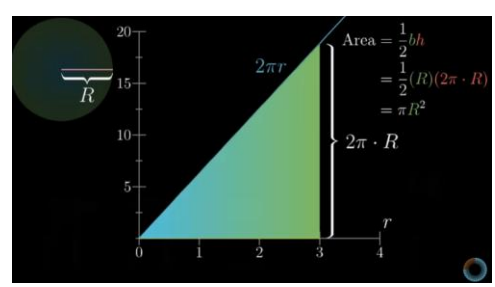

Figure: Area/Integral intuition used in derivative explanation.

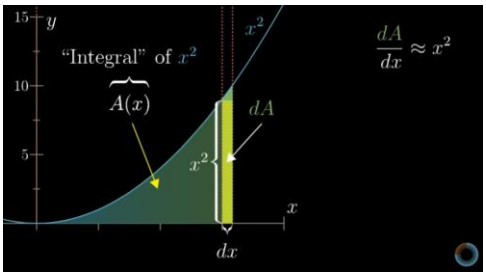

Figure: Derivative illustration — secant to tangent and limit.

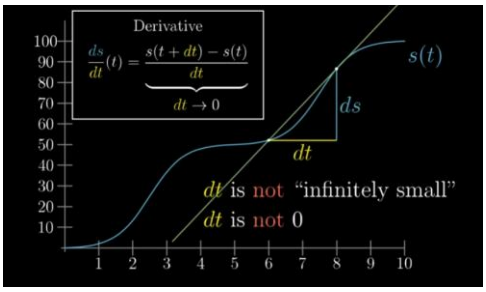

Figure: Additional derivative diagram.

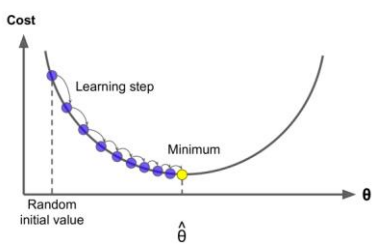

Figure: Gradient illustration on an example function.

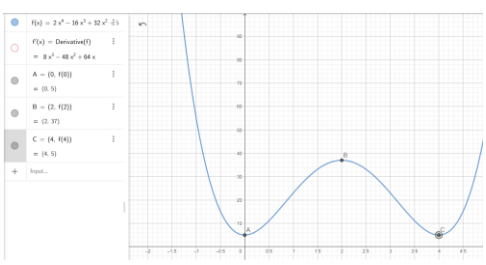

Figure: First-derivative criterion illustration.

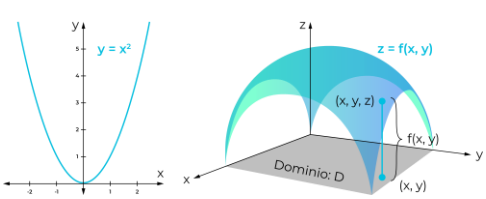

Figure: Supplementary visualization.